

# Введение в Requests.

Библиотека Requests в Python значительно упрощает процесс формирования HTTP-запросов. Трудно представить разработку парсера без использования этой библиотеки или её аналогов.

HTTP-запросы являются фундаментом Всемирной паутины. Каждый раз, когда вы открываете веб-страницу, ваш браузер отправляет на сервер множество запросов. В ответ сервер предоставляет все необходимые данные для отображения страницы. Только после этого ваш браузер может корректно отобразить веб-сайт для вашего просмотра.

С помощью запросов к HTML или API сайтов можно извлекать разнообразные данные: от информации о погоде и итогов спортивных состязаний до рейтингов фильмов, твитов, результатов поисковых систем, изображений и многого другого.

В этом разделе курса мы рассмотрим наиболее полезные методы библиотеки Requests. Научимся формировать запросы для различных сценариев, которые часто возникают при создании парсера. В частности, мы обсудим:

- Формирование запросов с использованием различных HTTP-методов;
- Редактирование заголовков запросов, параметров User-Agent, прокси и т. д.;
- Анализ данных запросов и ответов;
- Настройку запросов так, чтобы минимизировать риски возникновения нежелательных ситуаций.

```
import requests

response = requests.get('https://www.example.com')
print(response.text)
```

# Введение в Requests.

## Параметры запроса и передача данных

Когда дело доходит до передачи данных на сервер или запроса конкретной информации, **Requests** позволяет делать это легко и непринуждённо. С помощью параметров URL или тела запроса можно передавать данные в виде ключ-значение, что открывает широкие возможности для интерактивной работы с веб-ресурсами.

```
# Указываем параметры запроса в виде словаря
params = {'key1': 'value1', 'key2': 'value2'}
response = requests.get('https://www.example.com', params=params)
```

```
# Отправляем данные в формате словаря dict
```

```
data = {'name': 'John', 'age': 30}
response = requests.post('https://www.example.com/api', json=data)
```

## Аутентификация и безопасность

Важным аспектом при работе с HTTP-запросами является безопасность. **Requests** предлагает механизмы для аутентификации, которые защищают ваши запросы и данные. Это может быть базовая аутентификация с использованием логина и пароля или, например, более сложные механизмы с использованием токенов и ключей.

```
from requests.auth import HTTPBasicAuth

# Указываем логин и пароль
response = requests.get('https://www.example.com', auth=HTTPBasicAuth('user', 'pass'))
```

# Введение в Requests.

## Обработка исключений

Сетевая работа всегда подвержена различным ошибкам и исключениям. **Requests** предоставляет удобные средства для обработки этих моментов. От таймаутов и отсутствия сети до ошибок на стороне сервера — зная, как с этим работать, вы сможете создать устойчивые и надёжные парсеры.

```
try:
    response = requests.get('https://www.example.com', timeout=1)
except requests.Timeout:
    print("Слишком долгое ожидание!")
except requests.RequestException as e:
    print(f"Произошла ошибка: {e}")
```

## Работа с файлами и мультимедиа

Не менее интересной является возможность работы с файлами. Загрузка изображений, аудио, видео или даже больших данных для анализа — всё это можно сделать с помощью **Requests**. Это открывает двери для широкого спектра задач, начиная от автоматизации рабочих процессов и заканчивая сбором данных для научных исследований.

```
# Открываем файл и отправляем его на сервер
with open('file.txt', 'rb') as f:
    files = {'file': f}
    response = requests.post('https://www.example.com/upload', files=files)
```

# Преимущества Requests.

- **Простота и интуитивность:** Первое, что бросается в глаза при работе с Requests, — это чистота и понятность кода. Не требуется большого количества строк кода для выполнения даже сложных HTTP-запросов.
- **Функциональность:** Requests поддерживает множество методов HTTP, работу с куками, сессиями, заголовками и так далее. Эта библиотека предлагает решения для большинства задач, связанных с веб-скрапингом и API.
- **Безопасность и аутентификация:** Библиотека предоставляет встроенные механизмы для различных типов аутентификации, что делает ваш код не только короче, но и безопаснее.
- **Хорошая документация и сообщество:** Большое количество ресурсов, примеров и активное сообщество делают процесс изучения и работы с Requests намного проще.
- **Переиспользуемость кода:** Благодаря высокому уровню абстракции, код, написанный с использованием Requests, легко переиспользовать в других проектах.

# Недостатки Requests.

- **Не всегда подходит для асинхронного программирования:** Хотя существуют асинхронные форки и адаптации, сама по себе библиотека не предназначена для асинхронных операций (*aiohttp мы будем разбирать в модуле Асинхронный парсинг*). .
- **Зависимости:** Для некоторых специфичных задач, например, работы с WebSockets или определёнными типами аутентификации, может потребоваться установка дополнительных библиотек.
- **Потребление ресурсов:** При работе с большими объёмами данных или множеством одновременных соединений, Requests может быть менее эффективным по сравнению с некоторыми альтернативными решениями.
- **Недостаток некоторых продвинутых функций:** В то время как для большинства задач функционала Requests будет более чем достаточно, в некоторых случаях может потребоваться использование более специализированных библиотек.

# requests.get()

Метод `requests.get()` предназначен, как правило, для чтения данных, не изменяя их на сервере.

```
import requests

response = requests.get(url='http://httpbin.org/')
print(type(response))
```

|                              |                                                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>url</code>             | Передаёт ссылку - цель, куда будет отправлен запрос. (Обязательный)                                            |
| <code>params</code>          | Опциональный словарь или последовательность пар ключ-значение для отправки в строке запроса. (Необязательный)  |
| <code>headers</code>         | Опциональный словарь для настройки HTTP-заголовков. (Необязательный)                                           |
| <code>cookies</code>         | Опциональный словарь или объект <code>CookieJar</code> с куки, которые будут отправлены. (Необязательный)      |
| <code>auth</code>            | <b>AuthObject</b> для включения базовой аутентификации <b>HTTP</b> . (Необязательный)                          |
| <code>timeout</code>         | Число с плавающей запятой, описывающее тайм-аут запроса. (Необязательный)                                      |
| <code>allow_redirects</code> | Логическое значение. Установите значение <b>True</b> , если разрешено перенаправление. (Необязательный)        |
| <code>proxies</code>         | Протокол сопоставления словаря с URL-адресом прокси. (Необязательный)                                          |
| <code>stream</code>          | Удерживает соединение открытым, пока не получен весь <code>Response.content</code> (Необязательный)            |
| <code>data</code>            | Опциональный параметр для отправки данных в теле запроса (не рекомендуется для GET-запросов). (Необязательный) |
| <code>cert</code>            | Опциональный кортеж для указания пользовательского сертификата. (Необязательный)                               |

# Атрибуты объекта Response

Объект `Response` инкапсулирует всю информацию, возвращенную сервером, и предоставляет ряд атрибутов и методов для доступа к этой информации. От статус-кода HTTP до содержимого ответа, от заголовков до времени выполнения запроса — все эти данные доступны через различные атрибуты объекта `Response`.

Когда вы выполняете запрос, например:

```
response = requests.get('https://httpbin.org/get')
```

Переменная `response` теперь содержит объект `Response`, и вы можете получить доступ к различным атрибутам и методам этого объекта для изучения ответа.

Если бы можно было "распечатать" всю структуру объекта `Response`, она могла бы выглядеть примерно так:

```
Response {  
    status_code: 200,  
    headers: {...},  
    url: 'https://httpbin.org/get',  
    text: '...',  
    content: b'...',  
    json: <method>,  
    elapsed: <datetime.timedelta>,  
    ...  
}
```

*Можно обращаться к атрибутам или методам объекта, используя точечную нотацию*

# Атрибуты объекта Response

| Поле                     | Описание                                                            |
|--------------------------|---------------------------------------------------------------------|
| <code>status_code</code> | HTTP-код статуса ответа.                                            |
| <code>text</code>        | Текстовое представление содержимого ответа.                         |
| <code>content</code>     | Содержимое ответа в виде байтов.                                    |
| <code>json</code>        | Метод для десериализации JSON-ответа.                               |
| <code>headers</code>     | Заголовки HTTP, возвращаемые сервером.                              |
| <code>url</code>         | Исходный URL-адрес, на который был выполнен запрос.                 |
| <code>encoding</code>    | Кодировка ответа.                                                   |
| <code>elapsed</code>     | Время, затраченное на выполнение запроса.                           |
| <code>cookies</code>     | Куки, возвращаемые сервером.                                        |
| <code>history</code>     | Список объектов Response, представляющих историю перенаправлений.   |
| <code>ok</code>          | Логический атрибут, указывающий, был ли запрос успешным (коды 2xx). |
| <code>reason</code>      | Сообщение статуса HTTP (например, "OK", "Not Found").               |

# Атрибуты объекта Response

```
import requests

# URL для примеров
url = "https://httpbin.org/user-agent"

# Выполняем GET-запрос
response = requests.get(url)

# status_code: HTTP-код статуса ответа.
print("HTTP-код статуса ответа:", response.status_code)

# text: Текстовое представление содержимого ответа.
print("Текстовое содержимое ответа:", response.text)

# content: Содержимое ответа в виде байтов.
print("Содержимое ответа в виде байтов:", response.content)

# json: Метод для десериализации JSON-ответа.
json_response = response.json()
print("Десериализованный JSON-ответ:", json_response)

# headers: Заголовки HTTP, возвращаемые сервером.
print("Заголовки HTTP:", response.headers)
```

```
# url: Исходный URL-адрес, на который был выполнен запрос.
print("Исходный URL-адрес запроса:", response.url)

# encoding: Кодировка ответа.
print("Кодировка ответа:", response.encoding)

# elapsed: Время, затраченное на выполнение запроса.
print("Время выполнения запроса:", response.elapsed)

# cookies: Куки, возвращаемые сервером.
print("Куки, возвращаемые сервером:", response.cookies)

# history: Список объектов Response, представляющих историю перенаправлений.
print("История перенаправлений:", response.history)

# ok: Логический атрибут, указывающий, был ли запрос успешным (коды 2xx).
print("Запрос успешен (коды 2xx):", response.ok)

# reason: Сообщение статуса HTTP (например, "OK", "Not Found").
print("Сообщение статуса HTTP:", response.reason)
```

# Заголовки headers

Для имитации запроса от обычного веб-браузера в программировании на Python часто используется метод `.get()` библиотеки `requests` с именованным атрибутом `headers`. В этот атрибут передается словарь, в котором ключи — это названия HTTP-заголовков, а значения — это соответствующие значения этих заголовков.

При разработке парсеров рекомендуется отправлять свои заголовки вместе с GET-запросом. Это нужно потому, что сервер определяет, что делать с вашим запросом, именно на основе заголовков, которые вы отправили. Иногда требуется маскироваться под браузер, подменять **cookie**, **User-Agent**, **headers**, **IP-адрес** и прочее.

## Основные типы заголовков HTTP:

1. **Общие заголовки (General Headers):** Эти заголовки присутствуют и в запросах, и в ответах, но не относятся непосредственно к телу сообщения. Пример: `Cache-Control`, `Connection`.
2. **Заголовки запроса (Request Headers):** Отправляются только в HTTP-запросах. Они могут содержать информацию о ресурсе, который нужно получить, или о клиенте, который этот ресурс запрашивает. Примеры: `Accept`, `User-Agent`.
3. **Заголовки ответа (Response Headers):** Отправляются только в HTTP-ответах. Они обычно содержат информацию о сервере и о том, как интерпретировать тело ответа. Примеры: `Server`, `WWW-Authenticate`.
4. **Сущностные заголовки (Entity Headers):** Эти заголовки описывают тело сообщения, его тип и модификации. Примеры: `Content-Type`, `Content-Encoding`.

# Заголовки headers

## Почему это важно?

Многие веб-сервисы имеют механизмы защиты от автоматического доступа, например, от веб-скраперов или ботов. Один из способов обнаружения такого автоматического доступа — анализ HTTP-заголовков. Отправляя заголовки, которые обычно формируются веб-браузерами, можно "замаскировать" свою программу под обычного пользователя.

Таким образом выглядит стандартный питоновский `User-Agent`, это явно указывает на то, что запрос происходит от скрипта, а не от обычного веб-браузера. Это может привести к тому, что ваш IP-адрес будет заблокирован на сервере, особенно если речь идёт о сайтах с хорошо настроенными механизмами защиты от веб-скрапинга.

```
{  
  "user-agent": "python-requests/2.28.2"  
}
```

Этот `User-Agent` явно говорит серверу, что запрос был сделан с использованием библиотеки `requests` для Python, что часто является признаком автоматизированного доступа.

```
import requests  
  
# Определение заголовков, которые будут отправлены с запросом  
headers = {  
    'User-Agent': 'Mozilla/5.0',           # Идентификация типа браузера, который отправляет запрос  
    'Accept': 'text/html,application/xhtml+xml', # Типы контента, которые клиент может обработать  
    'Connection': 'keep-alive'           # Указание на необходимость использования постоянного соединения  
}  
  
# Выполнение GET-запроса с установленными заголовками  
response = requests.get('http://httpbin.org/user-agent', headers=headers)
```

# Заголовки headers

## Какие меры можно принять?

1. **Смена User-Agent:** одним из решений является использование списка различных `User-Agent` из заранее созданного файла и их случайный выбор при каждом запросе.
2. **IP-ротация:** В случае, если сервер блокирует по IP, можно использовать прокси-серверы для ротации IP-адресов.
3. **Задержки и ограничение частоты запросов:** Использование `time.sleep()` для введения задержек между запросами также может помочь избежать блокировки.
4. **Обработка капчи и других механизмов защиты:** В некоторых случаях может потребоваться распознавание и решение капчи.
5. **Реферер и другие заголовки:** Кроме `User-Agent`, можно также настроить другие HTTP-заголовки для имитации поведения настоящего пользователя.
6. **Сессии и куки:** Использование сессий и сохранение куки могут сделать ваш скрапинг менее подозрительным.
7. **Анализ JavaScript:** Некоторые сайты требуют выполнения JavaScript для доступа к контенту. В этом случае можно использовать библиотеки, которые могут интерпретировать JavaScript.

# Таймаут timeout

Когда мы отправляем запрос к сайту, наш парсер должен ожидать ответа от сервера, прежде чем продолжить выполнение. Если ваши запросы проходят через прокси-сервер, время ожидания может значительно увеличиться. По умолчанию, библиотека `requests` будет ждать ответа неопределённое время. Именно поэтому вы почти всегда должны задавать явный тайм-аут для каждого запроса.

`timeout` определяет максимальное время (в секундах), которое метод `.get()` будет ждать ответа от сервера.

`timeout` может быть числом (например, `timeout=1.5` ожидает 1.5 секунды) или кортежем для разделения времени на два этапа: подключение (`connect timeout`) и чтение (`read timeout`). Например, `timeout=(3.05, 27)` установит тайм-аут подключения в 3.05 секунды и тайм-аут чтения в 27 секунд.

Если установить `timeout=None`, `.get()` будет ждать ответа неопределённо долго. Это может быть рискованно, если сервер никогда не ответит или ответит с большой задержкой.

- Задание короткого тайм-аута может быть полезным для "быстрых" запросов, где задержка в ответе считается недопустимой.
- Для "медленных" запросов, например, когда ожидается большой объем данных, можно установить большое значение тайм-аута.

```
import requests
from requests.exceptions import ConnectTimeout

try:
    response = requests.get("http://example.com", timeout=5) # timeout в секундах
except ConnectTimeout:
    print("Connection to the server timed out.")
    # Здесь могут быть дополнительные действия, например, повторный запрос или логирование ошибки.
```

# Передача параметров в URL params

В работе с сайтами часто возникает необходимость передачи определённых данных в строке запроса URL. Если вы формируете URL вручную, то эти параметры обычно указываются в виде пар ключ/значение после знака вопроса.

## Что такое `params=` и для чего он используется?

Аргумент `params=` — это функциональность библиотеки Requests, которая позволяет передавать параметры в URL в виде словаря Python. Эти параметры автоматически конвертируются и добавляются к базовому URL, формируя таким образом полный URL запроса.

```
# Создание словаря с параметрами
params = {'key1': 'value1', 'key2': 'value2'}

# Отправка GET-запроса с параметрами
r = requests.get('https://httpbin.org/get', params=params)

# Вывод результирующего URL
print(r.url)

>>> https://httpbin.org/get?key1=value1&key2=value2
```

# Передача параметров в URL params

## Когда использовать params= ?

1. **Работа с API:** В большинстве случаев API требуют передачи определенных параметров для доступа к нужным данным.
2. **Динамические веб-сайты:** Для работы с динамическими веб-сайтами, где контент зависит от определённых переменных, параметры URL играют ключевую роль.

```
https://example.com/search?query=note&brand=Dell&min_price=50000&max_price=100000&sort=price_asc
```

3. **Пагинация:** При работе с веб-сайтами с пагинацией (разбиением контента на несколько страниц) параметры URL часто используются для указания конкретной страницы, с которой нужно собрать данные.

```
https://example.com/news?page=2
```

4. **Фильтрация данных:** Для демонстрации фильтрации данных рассмотрим пример с фиктивным API, предоставляющим информацию о фильмах. Предположим, что мы хотим получить список фильмов, выпущенных после определенного года и принадлежащих определенному жанру.

```
https://api.example.com/movies?year_after=2000&genre=action
```

# Прокси proxies

Библиотека Requests предоставляет возможность отправлять запросы, скрывая свой реальный IP-адрес. Зачем скрывать его? Например, при высокочастотных запросах есть риск получить бан.

**С точки зрения парсинга, протоколы HTTP и HTTPS имеют несколько ключевых различий:**

Шифрование:

- **HTTP (HyperText Transfer Protocol):** Не шифрует передаваемые данные, что делает их уязвимыми для перехвата и модификации.
- **HTTPS (HTTP Secure):** Использует SSL/TLS для шифрования данных, что повышает уровень безопасности.

Anti-Scraping и Captchas:

- Некоторые сайты могут использовать дополнительные меры безопасности на HTTPS, такие как более строгие правила по применению CAPTCHA или другие механизмы анти-скрапинга.

Заголовки и Cookies:

- Оба протокола передают заголовки и cookies, но HTTPS предоставляет дополнительную безопасность за счет шифрования.

Изменение Протокола:

- Некоторые сайты автоматически перенаправляют с HTTP на HTTPS. Это может потребовать дополнительной логики в вашем коде для корректной обработки.

Вы можете скачать бесплатные прокси вбив в google запрос "[Бесплатные прокси](#)".

# Статус коды

- Информационные 100 - 199
- Успешные 200 - 299
- Перенаправления 300 - 399
- Клиентские ошибки 400 - 499
- Серверные ошибки 500 - 599

|     |    |                                      |
|-----|----|--------------------------------------|
| 200 | OK | "Успешно". Запрос успешно обработан. |
|-----|----|--------------------------------------|

В 95% случаев нас интересует код с номером 200, т.к. он гарантирует, что ответ с сервера пришел полностью и без каких-либо ограничений.

Используя атрибут `status_code`, мы можем увидеть, какой код мы получили в ответ.

```
import requests

url = 'http://httpbin.org/'
response = requests.get(url)
print(response.status_code)

>>> 200
```

# Статус коды

Вот основные HTTP статус-коды, которые часто встречаются при работе с парсингом веб-сайтов:

|     |                                                                                                                               |
|-----|-------------------------------------------------------------------------------------------------------------------------------|
| 301 | Ресурс переехал на новый URL. Парсеру следует перейти по новому адресу.                                                       |
| 302 | Ресурс временно переехал. Парсер может перейти по новому адресу, но стоит помнить, что это перенаправление временное.         |
| 400 | Неверный запрос. Это может быть связано с неверно сформированным запросом или заголовками.                                    |
| 403 | Доступ к ресурсу запрещен. Сервер понял запрос, но он отказывается его выполнять. Обычно это связано с ограничениями доступа. |
| 404 | Ресурс не найден. Этот код часто встречается, если вы пытаетесь парсить несуществующую страницу.                              |
| 429 | Слишком много запросов. Это указывает на то, что вы превысили лимит запросов и должны подождать.                              |
| 500 | Внутренняя ошибка сервера. Это может быть связано с временными проблемами на сервере.                                         |
| 502 | Неверный шлюз. Обычно связано с проблемами на стороне сервера, которые парсер не может контролировать.                        |
| 503 | Сервис недоступен. Сервер временно не может обработать запросы, возможно, из-за перегрузки или технических работ.             |
| 200 | "Успешно". Запрос успешно обработан.                                                                                          |

# Содержимое ответа response

## Кодировка

По умолчанию, Requests автоматически определяет кодировку ответа сервера, основываясь на HTTP-заголовках. Однако, вы можете узнать, какую кодировку использует Requests, и при необходимости изменить её:

```
# Узнать текущую кодировку
print("Текущая кодировка:", r.encoding)
```

```
# Изменить кодировку
r.encoding = 'ISO-8859-1'
```

```
import requests
```

```
response = requests.get(url='https://jsonplaceholder.typicode.com/todos/')
print(response.json())
```

`r.json()`

```
>>> [{'userId': 1, 'id': 1, 'title': 'delectus aut autem', 'completed': False},
```

```
import requests
```

```
response = requests.get(url='https://jsonplaceholder.typicode.com/todos/')
print(response.text)
```

```
>>>[
  {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },.....
```

`.text`

# Содержимое ответа response

## `response.json()`

- **Тип данных:** Возвращает данные в формате, который может быть либо словарём, либо списком, в зависимости от структуры исходного JSON-объекта.
- **Обработка данных:** Автоматически декодирует JSON-строку в соответствующий Python-объект. Если строка не является валидным JSON, генерируется исключение `JSONDecodeError`.
- **Применение:** Используется, когда сервер возвращает JSON-форматированные данные.

## `response.text`

- **Тип данных:** Возвращает данные в виде обычной строки.
- **Обработка данных:** Не производит автоматическую обработку данных. Вам нужно самостоятельно парсить строку, если это необходимо.
- **Применение:** Используется, когда сервер возвращает данные в текстовом формате, который может быть не только JSON, но и HTML, XML и так далее.

# Содержимое ответа `response`

Свойство `response.content` предоставляет ответ сервера в виде байтовой строки. В отличие от `response.text`, который возвращает строку, и `response.json()`, который возвращает объект словарь или список, `response.content` предоставляет "сырые" байты ответа.

```
import requests

response = requests.get(url='http://httpbin.org/image/jpeg')
with open('image.jpeg', 'wb') as file:
    file.write(response.content)
```

## Когда использовать `response.content` ?

1. **Бинарные файлы:** Если вам нужно скачать или обработать бинарные файлы (изображения, аудио, видео и т. д.), `response.content` будет оптимальным выбором.
2. **Специфическая обработка данных:** Если вы хотите самостоятельно обработать "сырые" байты, например, для кастомной декодировки или преобразования.
3. **Дебаггинг:** Когда важно увидеть "сырой" ответ от сервера без какой-либо обработки.

# Работа с исключениями Exceptions

## В каких случаях необходимо заранее ловить исключения?

- При выполнении запросов к внешним веб-сервисам: необходимо обрабатывать исключения, чтобы правильно реагировать на проблемы соединения, тайм-ауты или неверные HTTP-ответы.
- При работе с динамически изменяемыми или пользовательскими URL: перехват исключений `URLRequired` помогает избежать ошибок из-за недопустимых URL.

## Какая практическая польза в исключениях?

- Исключения предлагают структурированный и организованный способ обработки ошибок, что способствует созданию более чистого и управляемого кода.
- Они позволяют разработчикам быстро идентифицировать и исправлять проблемы, улучшая качество программного обеспечения и уменьшая время на отладку.

# Работа с исключениями Exceptions

## Общие исключения:

- Когда вы перехватываете общее исключение, это как если бы вы ставили большую сеть, которая ловит все, что приходит ей под руку. Это может быть полезно, но в то же время, оно не предоставляет вам конкретной информации о том, что именно пошло не так.

Пример общего исключения:

```
try:
    response = requests.get('https://example.com')
except Exception as e:
    print(f"Произошла ошибка: {e}")
```

## Конкретные исключения:

- С другой стороны, когда вы перехватываете конкретное исключение, это как если бы вы использовали более мелкую сеть, которая целенаправленно ловит определенные типы ошибок и предоставляет вам более точную информацию о возникших проблемах.

Пример конкретного исключения:

```
try:
    response = requests.get('https://example.com')
except requests.ConnectionError as e:
    print(f"Произошла ошибка соединения: {e}")
```

# Работа с исключениями Exceptions

1. `requests.RequestException` : Это базовое исключение, от которого наследуются все другие исключения в библиотеке `requests` . Это амфиболическое исключение, которое может возникнуть во время обработки вашего запроса. Оно служит для перехвата всех исключений, генерируемых этой библиотекой.

**Частая причина возникновения:** Ошибки в обработке HTTP-запросов.

2. `requests.ConnectionError` : Это исключение возникает, когда происходит ошибка соединения, например, когда не удается подключиться к серверу или потеряно соединение в процессе выполнения запроса.

**Частая причина возникновения:** Проблемы с соединением, такие как отсутствие интернета или недоступность сервера.

3. `requests.HTTPError` : Это исключение генерируется, когда сервер возвращает ответ с HTTP-статусом, отличным от 2xx. Оно полезно для перехвата и обработки HTTP-ошибок.

**Частая причина возникновения:** Сервер возвращает код ошибки HTTP (например, 404 Not Found или 500 Internal Server Error).

4. `requests.MissingSchema` : Это исключение возникает, когда не предоставлен действительный URL для выполнения запроса. Другими словами, если URL отсутствует или недопустим, будет вызвано это исключение.

**Частая причина возникновения:** Недопустимый или отсутствующий URL.

# Работа с исключениями Exceptions

5. `requests.TooManyRedirects` : Это исключение возникает, когда запрос перенаправляется слишком много раз. Это может быть результатом бесконечного редиректа или превышения максимального количества разрешенных редиректов.

**Частая причина возникновения:** Слишком много перенаправлений, возможно, из-за циклического перенаправления.

6. `requests.ConnectTimeout` : Это исключение возникает, когда запрос не может быть выполнен из-за тайм-аута соединения с удаленным сервером. Запросы, которые вызвали это исключение, безопасно повторять.

**Частая причина возникновения:** Сервер не отвечает в течение установленного времени.

7. `requests.ReadTimeout` : Это исключение возникает, когда сервер не отправляет данные в отведенное количество времени. Это может произойти из-за проблем на стороне сервера или из-за недостаточно большого времени ожидания на стороне клиента.

**Частая причина возникновения:** Сервер не отправляет данные в отведенное время.

8. `requests.Timeout` : Это общее исключение для тайм-аутов, и его перехват будет перехватывать как `ConnectTimeout` , так и `ReadTimeout` . Это удобно, когда вы хотите обработать оба типа тайм-аутов одинаково.

**Частая причина возникновения:** Любой из вышеупомянутых тайм-аутов.

9. `requests.JSONDecodeError`: Это исключение возникает, когда текст не может быть декодирован в JSON. Это обычно происходит, когда сервер возвращает ответ, который не является действительным JSON-объектом, но ожидалось, что он будет JSON.

**Частая причина возникновения:** Ответ сервера не может быть декодирован как JSON.